Ders 10 Genişletilmiş Notu: Uygulama: Akbil Örneğiyle Kalıtım, Polymorphism, Yapıcı Sırası ve Abstract Sınıfa Giriş

Ders 10 Genişletilmiş Notu: Uygulama: Akbil Örneğiyle Kalıtım, Polymorphism, Yapıcı Sırası ve Abstract Sınıfa Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersten Önce: Bir Problemi Veri Yapısına Giydirmek
- 2. Akbil Senaryosu ve Person Üst Sınıfı
- 3. extends, super ve Yapıcıların Seçilmesi
- 4. Override: Her Sınıf Kendi Bakiyesini Günceller
- 5. Alt Sınıfa Özgü Alanlar ve toString
- 6. İlk Denemeler ve Erişim Belirteçleri
- 7. Üst Sınıf Tipinde Referans Dizisi
- 8. Yeni Bir Kategori: Emekli Sınıfı
- 9. Yaş ve Bakiye Birlikte: Döngüde Ortak ve Özel Metotlar
- 10. Statü Değiştirmek: Öğrenciyi Öğretmen Yapmak
- 11. Ters Yönde Atama Neden Olmaz?
- 12. Yapıcıların Çalışma Sırası ve Gizli super
- 13. NullPointerException ve Person p ile Student p Farkı
- 14. super.metot() ile Üst Sınıftaki Metodu Çağırmak
- 15. Abstract Metot ve Abstract Sınıf
- 16. Abstract Sınıftan Nesne Oluşturulamaz
- 17. Kodun Son Hâli
- 18. Örneği Geliştirme Önerisi

1. Dersten Önce: Bir Problemi Veri Yapısına Giydirmek

Ders başlamadan önce öğrencilerle başka bir derste karşılaşılan bir programlama sorusu konuşuldu. Soruda kodlanmış bir string veriliyordu. String okunurken bir rakam görülünce, önceki o kadar karakter siliniyordu: rakam 2 ise önceki 2 karakter, 3 ise önceki 3 karakter gider. En sonda kalan metin, çözülmüş mesajdır. Öğrenciler sorunun, doğrudan söylenmese de yığınla (stack) çözülmesinin beklendiğini anlattı: karakterler yığına atılır, silme gerektiğinde yığından çekilir.

Hocanın buradaki yorumu dersin geri kalanı için de geçerlidir: Stack'i kodlamak zor değildir, zor olan onu "probleme elbise gibi giydirmek"tir. Kuralları istediğiniz kadar öğrenin, soru değişince iş de değişir. Bu tür dersler kafayı çalıştırmayı gerektirir. Hoca, C ile yazarken de stack kullanmanın işi çok kolaylaştırdığını söyledi. Aksi hâlde diziyi fonksiyonlara göndermek, geri döndürmek ve pointer kullanmak gerekir.

2. Akbil Senaryosu ve Person Üst Sınıfı

Hoca, o gün teorik derste görülen kalıtım başta olmak üzere kavramları yerine oturtmak için basit bir soru çözmek istedi. Daha önce örnek olarak verdiği Akbil sistemini NetBeans'te canlı olarak kodladı. Kodu bir yere bakmadan yazdığını, öğrencilerin de önerebileceğini söyledi. Bu yüzden kod ders boyunca birkaç kez değişti.

Akbil sisteminde kullanıcılar vardır. Her kullanıcının ID'si, yaşı ve bakiyesi saklanır. Hoca önce ad ve soyadı da saymış, sonra bunlara gerek olmadığına karar vermiştir. Bu üç bilgi herkeste ortak olduğu için `Person` adlı bir sınıfa konur. Alanlar `private` tanımlanır, NetBeans'in `Insert Code` menüsüyle get/set metotları ve yapıcı üretilir.

Kullanıcıların bir de statüsü vardır: öğrenci (`Student`) ve öğretmen (`Teacher`). Bunlar `Person` sınıfından kalıtım alacaktır. Ders boyunca kurulan hiyerarşi düz metinle şöyledir:

```
Person  (ID, yas, bakiye; yasGuncelle, bakiyeGuncelle)
 |-- Student  (bolumAdi)
 |-- Teacher  (tecrube)
 |-- Emekli   (emekliYili; dersin ilerleyen kısmında eklendi)
```

2.1 Hangi Metot Nereye Yazılır?

Hoca metotların yerini şu soruyla belirledi: Bu işlem her alt sınıfta aynı mı, farklı mı?

Yaş güncelleme herkes için aynıdır. Öğretmenin de yaşı bir artar, öğrencinin de. Bu yüzden `yasGuncelle()` yalnız `Person` içinde yazılır, alt sınıflarda farklı biçimlerde yazılmaz. Hoca önce `void` yapmayı düşündü, sonra yeni yaşı döndürmesine karar verdi:

```
public int yasGuncelle() {
    this.yas++;
    return this.yas;
}
```

Bakiye güncelleme kullanıcıya göre değişir. Birinden 50 kuruş, birinden 1 lira alınır. Dolayısıyla üst sınıfta yapılacak bir bakiye güncellemesi bir anlam ifade etmez, çünkü asıl iş alt sınıflarda farklı şekillerde yapılacaktır. Hoca burada durup alt sınıflara geçti. Bu "anlamsız metot" meselesine dersin sonunda dönecek ve abstract kavramına oradan gelecektir.

3. extends, super ve Yapıcıların Seçilmesi

3.1 Yapıcısız Alt Sınıfın Hatası

`Student` ve `Teacher` sınıflarına `extends Person` yazılınca NetBeans şu hatayı gösterdi:

```
constructor Person in class Person cannot be applied to given types
```

Bu mesaj javac ile denendi, aynısı alınıyor. Hoca, hatayı `Teacher` sınıfına uygun bir yapıcı ekleyerek giderdi. Hatanın sebebi dersin ilerleyen kısmında görülecek gizli `super()` ile açıklanır (Bölüm 12): Yapıcı yazılmayan sınıf üst sınıfın parametresiz yapıcısını çağırmaya çalışır, oysa `Person`'da o anda yalnız parametreli yapıcı vardır.

3.2 super ile Üst Sınıfa Değer Göndermek

Hoca, `Teacher` yapıcısını elle yazdı. Yapıcı ID, yaş ve bakiye alır. Ama bu alanlar `Teacher`'a ait değildir, üst sınıftadır. O hâlde alınan değerler `super(...)` ile üst sınıfa gönderilir:

```
public Teacher(int ID, int yas, double bakiye) {
    super(ID, yas, bakiye);
}
```

NetBeans'te `Insert Code > Constructor` aynı işi kendisi yapar. Üst sınıfta birden fazla yapıcı varsa "hangi super constructor'ı kullanacaksın?" diye sorar. Hoca bu seçimi `Student` için kullandı.

3.3 Hangi Yapıcıya Gideceği Nasıl Belirlenir?

Soru şudur: `super(ID, yas, bakiye)` çağrısı, üst sınıftaki hangi yapıcıya gideceğini nereden biliyor? Cevap: Parametrelerin dizilimine ve birbirine dönüştürülüp dönüştürülememesine bakılır. `int, int, double` veriliyorsa ve `Person`'da doğrudan `int, int, double` alan bir yapıcı varsa o seçilir.

Hoca bunu göstermek için ID'nin tipini büyütmeye çalıştı. Önce `long int` yazdı ve "identifier expected" hatasını aldı. Java'da böyle bir tip yoktur (javac ile denendi, hata `<identifier> expected`). Sonra `long` ve `short` denedi. Bir `long` sürümü eklenince `Insert Code` iki farklı super constructor sundu.

Hoca, bazen iki yapıcının da uygun olabileceğini, derleyicinin "buna mı gideceğim, buna mı?" diye kalabileceğini söyledi. Bu konuya method overloading işlenirken örneklerle dönüleceğini belirtti. Burada yapılan iş constructor overloading'dir.

> **Ders sonrası not:** Hoca bu durum için "undefined behavior tarzı" dedi; doğrusu, Java'da bu bir derleme hatasıdır, program çalışmaz. Parametre tipleri `(long, int, double)` ve `(int, long, double)` olan iki yapıcıya `int, int, double` verilince `reference to P is ambiguous` hatası alınır (javac ile denendi; sınıf adı denemede `P` idi).

3.4 Eksik Bilgiyle Yapıcı

Hoca bir de bakiyesi bilinmeyen kullanıcı senaryosunu düşündü: ID ve yaş biliniyor, bakiye bilinmiyor. `Person` içine yalnız bu ikisini alan bir yapıcı yazılabilir. `Student`'ta `Insert Code` ile bu yapıcıya bağlanan bir yapıcı üretilebilir. Yani alt sınıf, üst sınıfın her alanını vermek zorunda değildir.

Aynı yerde hoca, daha önce bir öğrencinin sorduğu varsayılan parametreyi (default parameter) hatırladı ve "galiba yoktu" dedi.

> **Ders sonrası not:** Hocanın tereddüdünün cevabı: Java'da varsayılan parametre yoktur. `double c = 0.0` biçiminde parametre yazmak derleme hatası verir (javac ile denendi). Aynı ihtiyaç, burada olduğu gibi daha az parametreli bir yapıcı yazarak karşılanır.

4. Override: Her Sınıf Kendi Bakiyesini Günceller

Hoca `Person`'daki bakiye güncelleme metoduna görünür bir çıktı ekledi:

```
public double bakiyeGuncelle() {
    System.out.println("Person bakiye güncelleniyor");
    return 0.0;
}
```

Öğrencinin bakiyesi farklı, öğretmenin bakiyesi farklı güncellenecektir. Hoca "işte bu polymorphism oluyor" diyerek `Student` içine aynı imzalı metodu yazdı. NetBeans hemen `Add @Override Annotation` önerisini gösterdi. Hoca burada şu ayrımı yaptı: Metot adını küçük bir harf farkıyla yazarsanız hiçbir uyarı çıkmaz, çünkü o başka bir metottur. Aynı adla yazınca NetBeans "override etmeye çalışıyorsun, bunu ekle" der. Bu bir hata değil, uyarıdır.

Gövde şöyle kuruldu: bakiye `getBakiye()` ile alınır, ücret çıkarılır, yeni değer `setBakiye()` ile yazılır. Hoca önce 1 lira yazdı, sonra "öğrenciden o kadar kesmeyelim" diyerek 0,5 yaptı. Aynı metodu `Teacher`'a alıp 1,0 yaptı:

```
// Student
@Override
public double bakiyeGuncelle() {
    this.setBakiye(this.getBakiye() - 0.5);
    return this.getBakiye();
}

// Teacher
```

```java
@Override
public double bakiyeGuncelle() {
    this.setBakiye(this.getBakiye() - 1.0);
    return this.getBakiye();
}
```

4.1 Neden if Yazmıyoruz?

Hoca, öğrencilerin aklına gelebilecek soruyu kendisi sordu: “Bu kadar kolay bir şey için buna niye gerek var?” Başka yolla da yapılabilir. Nesnenin sınıf bilgisi alınır ya da statüsü tutulur. Öğrenciyse 0,5, öğretmense 1 lira çıkarılır. Bu çalışır.

Ama yazılım yaşayan bir şeydir. Üçüncü bir kategori eklemek gerekir, örneğin emekli. Sonra dördüncü, beşinci, belki beş yüzüncü. Uzun uzun `if` ya da `switch` yazmak mantıksız bir noktaya gider. Kalıtım ve polymorphism ile ortak alanlar üst sınıfta toplanır, her alt sınıf kendine özgü işi kendi içinde yapar.

5. Alt Sınıfa Özgü Alanlar ve toString

Alt sınıfların kendine ait bilgileri de olabilir. `Teacher` için kıdem yılı düşünüldü, adı `tecrube` oldu. Yapıcı dört parametreli hâle geldi. İlk üç parametre üst sınıfa gider, sonuncusu bu sınıfın alanına atanır:

```java
private int tecrube;

public Teacher(int ID, int yas, double bakiye, int tecrube) {
    super(ID, yas, bakiye);
    this.tecrube = tecrube;
}
```

`Student` için de aynı şekilde `String bolumAdi` eklendi.

Ardından `toString()` override edildi. Nesne kendini tanıtacaktır. Hoca `Teacher` içinde yazmaya başlarken `this.ID` diyemedi, çünkü `ID` alanı `Person`’da `private` tanımlıdır. Javac’ın bu durumdaki hatası `ID has private access in Person` şeklindedir (javac ile denendi). Çözüm `this.getID()` kullanmaktır.

5.1 protected Yapsak?

Hoca alanı `protected` yapınca `this.ID`’ye erişilebildiğini gösterdi, sonra “gerek var mı? Bence yok” dedi. Alanlara zaten get/set ile erişiliyor, değerler yapıcıyla atanıyor. Bu yüzden `private` olması daha iyidir, en azından her önüne gelen alanı değiştiremez.

5.2 final Sınıf

Aynı anda bir yan bilgi gösterildi: `Person` sınıfı `final` yapılırsa ondan kalıtım alınamaz. Derleyici `cannot inherit from final Person` der (javac ile denendi). Hoca, C#’ta bunun karşılığının `sealed` olduğunu ekledi.

5.3 toString Metodunun Yazılması

Hoca `Insert Code` menüsünde hazır bir `toString` aradı, bir öğrencinin IntelliJ’de bunun olduğunu söylediğini hatırladı.

Soru: Override kısmında bir `Person` bir de `Object` görünüyor, orada yok mu? **Cevap:** Hoca `Object`’i açtı ama o anda aradığı `toString` listede görünmedi, elle yazmaya karar verdi. Bölüm 8’de `Emekli` sınıfında aynı menüde `toString` görünecektir.

Bir öğrenci de tecrübe değerinin bitişik çıkmaması için etiketin sonuna eşittir koymayı önerdi, hoca bunu uyguladı. Son biçim şöyledir:

```java
@Override
public String toString() {
    return "[ID=" + this.getID() + ", YAS=" + this.getYas()
         + ", BAKIYE=" + this.getBakiye() + ", TECRUBE=" + this.tecrube + ".]";
}
```

`Student`'ta aynı metot `bolumAdi` ile yazıldı.

6. İlk Denemeler ve Erişim Belirteçleri

Hoca test sınıfında önce bir öğrenci nesnesi oluşturdu. `Student`'ın yapıcısına uygun olarak ID 1, yaş 15, bakiye 15.0, bölüm "Bilgisayar" verildi. Sonra `s1.bakiyeGuncelle()` sonucu "yeni bakiye" diye yazdırıldı.

> **Ders sonrası not:** Hoca burada "bakiye güncelle bir integer değer döndürüyor" dedi; doğrusu, metot `double` döndürür.

İlk çalıştırmada beklenen değer çıkmadı. Sebep, `Student`'taki metodun o anda hâlâ `return 0.0;` döndürmesiydi. Hoca bunu `return this.getBakiye();` olarak düzeltti.

Bundan sonraki bir süre NetBeans'in son değişiklikleri çalıştırmamasıyla uğraşıldı. Hoca `Clean and Build` yapınca ve `Run File` ile çalıştırınca sorun aşıldı, "en son derlenmiş hâlini çalıştırıyor nedense, tekrar derlemek gerekiyor galiba" dedi. Proje ayarlarındaki `Compile on Save` seçeneğinin bazı öğrencilerin yaşadığı sorunla ilgili olabileceğini de söyledi.

Sonuçlar (python ile hesaplandı):

- **Öğrenci:** `15.0 - 0.5 = 14.5`. Derste de "yeni bakiye 14.5" görüldü.
- **Öğretmen:** ID 1, yaş 40, bakiye 15.0, tecrübe 20 ile oluşturuldu (ID sonra 2 yapıldı). `15.0 - 1.0 = 14.0`.

Hoca, `Teacher`'da bakiye alanı olmamasına rağmen `t1` üzerinden `getBakiye()` çağrılabildiğine dikkat çekti: `t1`, `Person` sınıfından kalıtım almıştır.

Sonra get/set metotlarının erişim belirtecine geldi. Bazı metotlar dışarıya açık, bazıları kapalı olmalıdır. Hoca, `getBakiye` ve `setBakiye` `protected` yapılsaydı bunlara ana sınıftan erişilemeyeceğini, yalnız alt sınıflardan erişilebileceğini söyledi.

> **Ders sonrası not:** Hoca `protected` metoda ana sınıftan erişilemeyeceğini söyledi; doğrusu, `protected` üyeye aynı paketteki bütün sınıflar da erişebilir. Erişim yalnız başka paketteki ilişkisiz sınıflar için kapanır. Javac ile denendi: aynı pakette derleniyor, başka paketten çağırınca `getBakiye() has protected access in Person` hatası alınıyor. Hocanın sözü, ana sınıf başka bir paketteyse doğrudur.

7. Üst Sınıf Tipinde Referans Dizisi

Hocanın "işin daha güzel tarafı" dediği kısım budur. `Person` tipinde bir dizi oluşturulur:

```java
Person[] kisiler = new Person[10];
```

Bu satır 10 nesne üretmez, 10 **referans** üretir. Daha önce kitaplar için yapılan örnekteki gibi, her konuma ayrıca `new` ile üretilmiş bir nesne atanmalıdır. Atanan nesne `Student` da olabilir, `Teacher` da:

```java
kisiler[0] = s1;
kisiler[1] = t1;
kisiler[2] = new Teacher(3, 55, 25.0, 20);
kisiler[3] = new Student(4, 22, 30.0, "Elektronik");
```

Hoca ID'leri 1, 2, 3, 4 olacak şekilde düzeltti. Bir öğrenci de son satırda indisin yanlışlıkla yine 2 yazıldığını fark etti, 3 yapıldı.

Sonra döngü yazıldı. Dizinin her elemanı `Person` türünden referans içerdiği için döngü değişkeni de `Person` olur:

```java
int i = 1;
for (Person kisi : kisiler) {
```

```
    System.out.println(i++ + "---" + kisi);
}
```

Hoca önce döngüde `kisi.bakiyeGuncelle()` sonucunu yazdırdı. Kişi öğrenciyse `Student`'taki, öğretmense `Teacher`'daki metot çalışır. Sonra bunun yerine kişinin kendini tanıtmasını istedi. `toString`'i çağırmak için nesnenin referansını vermek yeterlidir. Sonuna `.toString()` yazmaya gerek yoktur. Çalışma anında `Student` ise `Student`'taki, `Teacher` ise `Teacher`'daki `toString` çalışır.

İlk çıktıda öğrenci satırında tuhaf bir şey göründü: tecrübe etiketinin yanında "Bilgisayar" yazıyordu. `Teacher`'dan kopyalanan `toString`'te etiket değişmemişti. Hoca etiketi bölüm olarak düzeltti.

> **Ek bilgi (derste anlatılmadı):** Çağrılacak metodun derleme anında referans tipine göre değil, çalışma anında nesnenin gerçek sınıfına göre seçilmesi, Java literatüründe dinamik bağlama (dynamic binding) adıyla geçer. Sonraki teorik derste büyük olasılıkla bu adla ele alınacak.

8. Yeni Bir Kategori: Emekli Sınıfı

Hoca polymorphism'in kazancını şöyle gösterdi: Yeni bir kategori eklemek için `Student`, `Teacher` ve `Person`'a hiç dokunulmaz. Yeni bir `Emekli` sınıfı açılır, kendine ait bir `emekliYili` alanı olabilir, `extends Person` denir. `Insert Code > Constructor` ile üst sınıftan yapıcı alınır.

`Insert Code > Override Method` ise override edilebilecek metotları listeler: `Person`'dan `bakiyeGuncelle`, `yasGuncelle` ve get/set metotları, daha üst sınıf olan `Object`'ten de `toString`. Hoca bu kez listede `toString`'i buldu ve seçti. NetBeans gövdeye üst sınıftaki `toString`'i çağıran bir satır koydu. Hoca bu satırı "Ben emekliyim, benden para almayın" metniyle değiştirdi.

> **Ders sonrası not:** Hoca üretilen `super.toString()` için "Person'da tanımlıysa onu yapacak, Person'da da tanımlıysa Object'teki çağrılacak" dedi; doğrusu ikinci kısım "tanımlı değilse"dir. Üst sınıfta `toString` yoksa `Object`'teki çalışır (javac ile denendi, çıktı `Emekli@...` biçiminde başlıyor).

9. Yaş ve Bakiye Birlikte: Döngüde Ortak ve Özel Metotlar

Hoca döngüyü, üst sınıftan gelen metotla alt sınıftaki metodu yan yana gösterecek şekilde değiştirdi. Önce yaş güncellenir ve kişi yazdırılır, sonra bakiye güncellenir ve kişi yine yazdırılır:

```
int i = 1;
for (Person kisi : kisiler) {
    kisi.yasGuncelle();                    // herkes için Person'daki metot
    System.out.println(i++ + "---" + kisi);
    kisi.bakiyeGuncelle();                 // nesnenin gerçek sınıfındaki metot
    System.out.println(i + "---" + kisi);
}
```

Dört kişi için bakiye hesabı aşağıdadır (python ile hesaplandı, Java çıktısıyla da aynı):

Kişi	Sınıf	Yaş	Yeni yaş	Bakiye	Ücret	Yeni bakiye
1	Student	15	16	15.0	0.5	14.5
2	Teacher	40	41	15.0	1.0	14.0
3	Teacher	55	56	25.0	1.0	24.0
4	Student	22	23	30.0	0.5	29.5

Yaş güncellemesi herkes için aynı biçimde üst sınıftan, bakiye güncellemesi alt sınıftan çalışır.

> **Ders sonrası not:** Hoca çıktıyı yorumlarken "birinin bakiyesini 0,5 artırırken birini 1 artırıyor" dedi; doğrusu, bakiye azalıyor. Metotlar ücreti bakiyeden çıkarır.

Soru: Normal bir dizinin elemanlarının bellekte sabit bir boyutu vardır. Ama `Teacher` ile `Student` nesneleri farklı boyutta değil mi? Bunlar aynı dizide nasıl duruyor? **Cevap:** Evet, nesnelerin boyutu farklıdır. Ama bu dizi nesnelerin kendisini değil, `Student` ya da `Teacher` nesnesinin **referansını** saklar. Doğrudan adres saklanır. Üst sınıf `Person` olduğu için bu referanslar `Person` tipinde bir referans dizisinde tutulabilir.

10. Statü Değiştirmek: Öğrenciyi Öğretmen Yapmak

Referansın üst sınıf tipinde olması başka bir olanak da verir. Bir `Person` referansıyla öğrenci oluşturulabilir. Hocanın deyişiyle "person gibi başladık, öğrenci gibi bitirdik":

```
Person p = new Student(5, 15, 10.0, "Bilgisayar");
```

Şimdi bu öğrencinin öğretmen olduğunu düşünelim. Öğrenciyi öğretmen ya da emekli yapmak, yani statüsünü değiştirmek mümkündür. Bunun için yeni bir `Teacher` nesnesi oluşturulur, ama eski bilgiler kaybedilmemelidir. Ortak bilgiler `p` üzerinden alınıp yeni nesnenin yapıcısına verilir:

```
p = new Teacher(p.getID(), p.getYas(), p.getBakiye(), 0);
```

Tecrübesi henüz olmadığı için son parametre 0'dır. Öğrencinin bölüm bilgisi gider, çünkü `Teacher`'da böyle bir alan yoktur. Artık `p` bir öğretmen gibi davranır.

Hoca bunu çıktıda görmek isterken beklediği sonucu ilk anda alamadı, "hata yaptık mı burada?" diyerek diziye atama satırının yerini değiştirdi. Sonunda ID'si 5, yaşı 16, tecrübesi 0 olan kişi öğretmen olarak listelendi.

> **Ders sonrası not:** Sıra burada önemlidir. `kisiler[4] = p;` satırı `p = new Teacher(...)` satırından önce yazılırsa, dizinin dördüncü indisi hâlâ eski `Student` nesnesini gösterir. Atama yalnız `p` referansını değiştirir, dizideki referansı değiştirmez. Javac ile denendi: `kisiler[4]` için `BOLUM=Bilgisayar`, `p` için `TECRUBE=0` yazdırılıyor. Atama yeni nesneden sonra yapılınca dizi de öğretmeni gösterir.

Bu çalışma sırasında dizi boyutu 10 olduğundan, boş konumlar yüzünden çıktının sonunda `NullPointerException` da görüldü. Hoca bunu şimdilik önemsemedi ve diğer satırları görebilmek için boyutu geçici olarak 5 yaptı. Konuya Bölüm 13'te dönüldü.

11. Ters Yönde Atama Neden Olmaz?

Hoca bir öğrencinin sorusunu ele aldı: Üst sınıf nesnesi alt sınıf referansına atanabilir mi?

```
Student pp = new Person(1, 2, 3);
```

Derleyici şu hatayı verir:

```
incompatible types: Person cannot be converted to Student
```

Hiyerarşinin tersine dönüşüm yapılamaz. Hoca bunu sayısal tiplere benzetti: `int`'ten `long`'a dönüşüm kendiliğinden olur, `long`'dan `int`'e olmaz. Orada doğrudan kalıtım olmadığını, ama "aynen bunun gibi düşünmek" gerektiğini söyledi. Javac'ta ikinci durumun hatası `possible lossy conversion from long to int` şeklindedir (javac ile denendi).

> **Ders sonrası not:** Hoca bunu sözle "alt sınıftan üst sınıfa cast edemiyoruz" diye, kod yorumuna da "child to parent conversion" diye yazdı; doğrusu ters yöndür. Burada üst sınıf nesnesi alt sınıf referansına atanıyor, yani hata parent'tan child'a dönüşümdedir. Alt sınıf nesnesini üst sınıf referansına atamak (`Person p = new Student(...)`) serbesttir. `Student` referansı `bolumAdi` gibi üyeler vaat eder, `Person` nesnesinde ise bunlar yoktur. `new Person(1, 2, 3)`'ü `Student` referansına atamanın hatası javac ile denendi.

12. Yapıcıların Çalışma Sırası ve Gizli super

Hoca, kalıtımla ilgili çok başka bir kavram kalmadığını söyleyip bir öğrencinin sorusuna cevap olarak yapıcıların sırasını gösterdi. Bunun için `Student` ve `Person`'a parametresiz yapıcılar eklendi:

```java
public Person() {
    System.out.println("Person oluşturuldu");
}

public Student() {
    System.out.println("Öğrenci oluşturuldu");
}
```

`new Student()` yazılınca olanlar hocanın anlatımıyla şöyledir:

1. Nesne oluşturulmadan önce `Student`'ın üst sınıfına bakılır. Üst sınıf `Person`'dır.
2. Önce static üyeler varsa onlar hazırlanır.
3. `Person`'ın üye değişkenleri (member variable) oluşturulur. Başka bir sınıf tipinde değişken varsa o nesne de bu aşamada oluşturulur.
4. `Person`'ın yapıcısı çalışır, "Person oluşturuldu" yazılır.
5. `Student`'ın boş yapıcısı çalışır, "Öğrenci oluşturuldu" yazılır.
6. Nesne tamamen oluşturulmuştur, referans ataması yapılır.

Çalıştırınca çıktı şöyledir (javac ile denendi):

```
Person oluşturuldu
Öğrenci oluşturuldu
```

Oysa `Student()` içinde üst sınıfın yapıcısı çağrılmamıştır. Hocanın açıklaması: Burada gizli bir `super` vardır. Yazsanız da yazmasanız da oradadır ve varsayılan olarak boş parametreli yapıcıyı çağırır. Bölüm 3.1'deki hatanın sebebi de budur.

> **Ek bilgi (derste anlatılmadı):** Hocanın sıralaması bir denemeyle görülebilir. `Person` ve `Student`'a static blok ve metotla ilklenen alan eklenip iki kez `new Student()` yazıldı. Javac ile denendi, ilk nesnede sıra şöyledir: `Person static`, `Student static`, `Person alan`, `Person yapıcı`, `Student alan`, `Student yapıcı`. İkinci nesnede static satırlar yoktur, çünkü static kısım sınıf için bir kez çalışır.

13. NullPointerException ve Person p ile Student p Farkı

Soru: Çıktının sonundaki `NullPointerException` nedir? **Cevap:** Dizi önce 10 elemanlıydı, sonra 6 yapıldı. Döngü dizinin boyutuna göre döner. Altıncı elemana hiçbir referans atanmadığı için orası `null`'dur. `null` üzerinden yazdırılacak bir şey bulunamadığından exception fırlatılır. Hoca, ileride exception konusunda bu hatayı nasıl ele alacaklarını göstereceğini söyledi. Javac ile denendi: Dört kişi ve `p` atandıktan sonra `new Person[6]` ile son konumda bu hata alınıyor.

Soru: `Student p = new Student(...)` ile `Person p = new Student(...)` arasında ne gibi bir kullanım farkı var, neden böyle yapıyoruz? **Cevap:** Üst sınıf `Person`, alt sınıflar `Student`, `Teacher` ve `Emekli`'dir. Bu üçü, üst sınıf tipinde ortak bir değişkenle tanımlanabilir. **Nesnenin kendisinde herhangi bir değişiklik yoktur, farklı olan nesneyi tutan referans değişkendir.**

Bunun kazancı şudur: Diyelim ki 100 öğrenci ve 500 öğretmen var, toplam 600 kişi. Hepsi `Person`'dan kalıtım almıştır. Öğrencileri ayrı, öğretmenleri ayrı çağırmak yerine `Person` tipinde referanslar tutulur, tek döngüyle bütün elemanlar dolaşılır. Örneğin herkese 5 lira bonus yüklenecekse, tek tek uğraşmak yerine bu döngü kullanılır. Yapılan işe "upcasting gibi bir şey" denir, yani üst sınıf üzerinden erişmek.

İkinci kazanç önceki bölümdeki statü değişikliğidir. `p`, `Student` tipinde tanımlansaydı, `Teacher` nesnesi ona atanamazdı. Çünkü bunlar birbirine dönüşümü mümkün olmayan, eşit düzeyde sınıflardır. Hata `incompatible types: Teacher cannot be converted to Student` şeklindedir (javac ile denendi).

14. super.metot() ile Üst Sınıftaki Metodu Çağırmak

Hoca dersin son konusuna şu rahatsızlıkla girdi: `Person`'daki `bakiyeGuncelle()` boşuna yazılmıştır. Alt sınıfların hepsi onu override ettiği için "Person bakiye güncelleniyor" yazısı hiç görülmedi.

Önce `Student`'taki metoda "STD bakiye güncelleniyor" çıktısı eklendi. Sonra üst sınıftaki metot çağrıldı. Hoca önce `this.super` denedi, "yok bu değildi" diyerek `super.bakiyeGuncelle()` yazdı. `this.super` Java'da geçerli değildir (javac ile denendi):

```
@Override
public double bakiyeGuncelle() {
    super.bakiyeGuncelle();
    System.out.println("STD bakiye güncelleniyor");
    this.setBakiye(this.getBakiye() - 0.5);
    return this.getBakiye();
}
```

Bunun normal bir metot çağırmaktan hiçbir farkı yoktur. Kurgu gerektiriyorsa, yani işin bir kısmı üst sınıfta ortak, kalanı öğrenciye ya da öğretmene göre yapılacaksa bu yol kullanılır. Çıktıda önce üst sınıfın, sonra alt sınıfın satırı görünür:

```
Person bakiye güncelleniyor
STD bakiye güncelleniyor
```

Hoca, teorik derste konuşulan bir kısıtı burada denemek istedi. Çağrıyı metot içinde başka bir satıra taşıdı ve hata beklerken hata almadı. "Burada mümkünmüş" dedi ve bunun yapıcıdaki kadar katı olmadığını ekledi. Yani ilk satırda olma kısıtı yapıcıdaki `super(...)` içindir, normal metottaki `super.metot()` için değil. Javac ile denendi: Çağrı aşağı taşınınca satırların sırası da yer değiştiriyor.

> **Ders sonrası not:** Bu kısıt dersin yapıldığı Java sürümleri için geçerlidir. Javac 25 ile `-source 16` ve `-source 21` verilince yapıcıda `super()`'dan önce satır yazmak `flexible constructors is not supported` hatası veriyor. JDK 25'in varsayılan ayarında ise aynı kod derleniyor (javac ile denendi).

Hoca, `this.bakiyeGuncelle()` yazılsa da "yine aynısının çalışacağını" söyledi, bunu çalıştırıp göstermedi.

> **Ders sonrası not:** Bu çağrı, metodun kendi içinde yazıldığında üst sınıftaki metodu değil yine aynı metodu çağırır ve durmadan kendini çağırır. Javac ile derleniyor, çalıştırınca `StackOverflowError` alınıyor (denendi). Üst sınıftaki metot için `super.` gerekir.

15. Abstract Metot ve Abstract Sınıf

Hoca şimdi asıl önemli konuya geldi. Varsayalım ki `Person`'daki `bakiyeGuncelle()` hiç önemli değil, burası çalışmayacak. Gövdeyi boş bırakmak isteniyor, `return` da yazılmak istenmiyor. Ama iki engel var:

- `return` yazılmazsa kod hata verir. Javac mesajı `missing return statement` olur (javac ile denendi).
- Metot `void` de yapılamaz, çünkü alt sınıflarda `double` yazılmıştır. Javac mesajı `return type double is not compatible with void` olur (javac ile denendi).

Çözüm, metodun önüne `abstract` yazmaktır. NetBeans bu kez gövdenin silinmesini ister (javac mesajı `abstract methods cannot have a body`). Gövde silinip sonuna noktalı virgül konunca gövdesiz bir metot elde edilir:

```
public abstract double bakiyeGuncelle();
```

Bu kez sınıf satırında hata çıkar: Abstract metotları yalnız abstract sınıflar içerebilir. Javac bunu `Person is not abstract and does not override abstract method bakiyeGuncelle() in Person` diye bildirir (javac ile denendi, metot adı denemede kısaydı). Sınıf da abstract yapılınca sorun kalkar:

```
public abstract class Person { ... }
```

15.1 Yaygın Bir Yanlış Önerme

Hoca bir önermeyi test etti: Bir sınıf abstract ise en az bir abstract metodu olmalı mıdır? Abstract metodu kapatıp sınıfı `abstract` bıraktı ve hata almadı. Javac ile de denendi, derleniyor.

> **Hocanın vurgusu:** “Sınıf abstract ise en az bir abstract metodu olması lazım” diye anlatılır, ama bu yanlış bir önermedir. Doğrusu bunun tersidir: Bir sınıfta en az bir abstract metot varsa, o sınıf abstract olarak tanımlanmak zorundadır.

15.2 Abstract Metot Bir Standart Koyar

Bu noktaya içi boş bir metot istediğimiz için geldik. Ama abstract metot boşluktan fazlasını yapar: Bundan kalıtım alan bütün sınıflara yön belirler. “Bu bir standarttır, `bakiyeGuncelle`’yi kendinde bulunduracaksın” der. Hocanın benzetmesi Avrupa Birliği’ne giriştir: Girmek için şunları şunları yerine getirmen gerekir denir. Abstract metot da aynen böyledir.

Hoca bunu görmek için `Student`’taki `bakiyeGuncelle`’yi kapattı ve şu hatayı aldı (javac ile denendi):

```
Student is not abstract and does not override abstract method bakiyeGuncelle() in Person
```

NetBeans bunun için `Implement all abstract methods` önerisini sunar. Üst sınıftaki abstract metotları alt sınıfa kendisi ekler, `@Override`’ı da koyar. Hoca, gövdenin otomatik içerik yerine kendi kurgunuza göre değiştirilmesi gerektiğini söyledi. Abstract konusunu gelecek hafta teorik olarak daha ayrıntılı göreceklerini ve bunun ötesinde ufak tefek ayrıntılar olduğunu ekledi.

Hoca geldikleri yolu şöyle özetledi: Bakiye güncelleme öğrencide de öğretmende de var, üst tarafta boşuna duruyor. İçi boş olsun istiyoruz, ama `return` yazmak da istemiyoruz. Metodu abstract yaptık. En az bir abstract metot olunca sınıfı da abstract yapmak zorunda kaldık.

16. Abstract Sınıftan Nesne Oluşturulamaz

Son olarak hoca farkı göstermek için kodu abstract öncesi hâline getirmeye başladı. Bu sırada `Emekli`, abstract metodu yazmadığı için hata verdi. Metot kaldırılınca da `@Override` satırları hata verdi, çünkü override edilecek metot kalmamıştı. Javac bunun için `method does not override or implement a method from a supertype` der (javac ile denendi). Hoca bu satırları kapattıktan sonra `Person`’dan nesne üretti:

```
Person p4 = new Person();
```

Bu hâliyle hata yoktur. `Ctrl+Z` ile sınıf yeniden abstract yapılınca aynı satır hata verir:

```
Person is abstract; cannot be instantiated
```

> **Hocanın vurgusu:** Abstract sınıfların en önemli özelliklerinden biri, ondan nesne (örnek) alınamamasıdır.

Ama referans olarak kullanmak serbesttir. `Person` bir referanstır, nesne `Student` ya da `Teacher` olabilir:

```
Person a = new Student(1, 15, 15.0, "Bilgisayar");
Person b = new Teacher(2, 40, 15.0, 20);
```

Hoca kodda bu iki hatalı satırı yorumla işaretledi. `new Person()` için “illegal because it is abstract class” yazdı. “Bu da” diyerek ikinci açıklamayı ekledi: “due to the error of child to parent conversion”. Sözle de küçük bir sınıftan büyük bir sınıfa dönüşüm yapılamayacağını söyledi. Bu ifadenin neden ters olduğu Bölüm 11’deki notta açıklandı.

Soru: `bakiyeGuncelle()` üst sınıftan tamamen silinirse ne olur? **Cevap:** Alt sınıflarda yine istenildiği gibi yazılabilir, çünkü artık override edilmesi gereken bir metot değildir. Ama döngüde `kisi.bakiyeGuncelle()` çağrısı üzerinden ilgili alt sınıfın metoduna gitmek, yani polymorphism kaybolur. Bu metot hem bir standart sunar hem de nesnenin alt sınıfına göre doğru metodun çağrılmasını sağlar. Silinirse olmaz. Javac ile denendi: `Person` tipindeki referansta çağrı `cannot find symbol` hatası veriyor.

17. Kodun Son Hâli

Aşağıdaki kod bütün bölümlerin birleşimidir. Javac ile derlenip çalıştırıldı.

Ders sonrası not: `Emekli`'nin `bakiyeGuncelle()` gövdesi derste gösterilmedi. `Person` abstract olunca bu metot zorunlu olduğu için derleme amacıyla, bakiyeyi değiştirmeden döndüren bir gövde yazıldı. Hocanın `toString` metnindeki "benden para almayın" sözüne uyar.

```
package Hafta5;

public abstract class Person {
    private int ID;
    private int yas;
    private double bakiye;

    public Person() {
        System.out.println("Person oluşturuldu");
    }

    public Person(int ID, int yas, double bakiye) {
        this.ID = ID;
        this.yas = yas;
        this.bakiye = bakiye;
    }

    public Person(int ID, int yas) {
        this.ID = ID;
        this.yas = yas;
    }

    public int getID() { return ID; }
    public int getYas() { return yas; }
    public double getBakiye() { return bakiye; }
    public void setBakiye(double bakiye) { this.bakiye = bakiye; }

    public int yasGuncelle() {
        this.yas++;
        return this.yas;
    }

    public abstract double bakiyeGuncelle();
}
```

```
package Hafta5;

public class Student extends Person {
    private String bolumAdi;

    public Student() {
        System.out.println("Öğrenci oluşturuldu");
    }

    public Student(int ID, int yas, double bakiye, String bolumAdi) {
        super(ID, yas, bakiye);
        this.bolumAdi = bolumAdi;
    }

    @Override
    public double bakiyeGuncelle() {
        this.setBakiye(this.getBakiye() - 0.5);
        return this.getBakiye();
    }

    @Override
```

```
    public String toString() {
        return "[ID=" + this.getID() + ", YAS=" + this.getYas()
             + ", BAKIYE=" + this.getBakiye() + ", BOLUM=" + this.bolumAdi + ".]";
    }
}
```

```
package Hafta5;

public class Teacher extends Person {
    private int tecrube;

    public Teacher(int ID, int yas, double bakiye, int tecrube) {
        super(ID, yas, bakiye);
        this.tecrube = tecrube;
    }

    @Override
    public double bakiyeGuncelle() {
        this.setBakiye(this.getBakiye() - 1.0);
        return this.getBakiye();
    }

    @Override
    public String toString() {
        return "[ID=" + this.getID() + ", YAS=" + this.getYas()
             + ", BAKIYE=" + this.getBakiye() + ", TECRUBE=" + this.tecrube + ".]";
    }
}
```

```
package Hafta5;

public class Emekli extends Person {
    private int emekliYili;

    public Emekli(int ID, int yas, double bakiye) {
        super(ID, yas, bakiye);
    }

    @Override
    public double bakiyeGuncelle() {
        return this.getBakiye();
    }

    @Override
    public String toString() {
        return "Ben emekliyim, benden para almayın.";
    }
}
```

```
package Hafta5;

public class Main1 {
    public static void main(String[] args) {
        Student s1 = new Student(1, 15, 15.0, "Bilgisayar");
        Teacher t1 = new Teacher(2, 40, 15.0, 20);

        Person[] kisiler = new Person[6];
        kisiler[0] = s1;
        kisiler[1] = t1;
        kisiler[2] = new Teacher(3, 55, 25.0, 20);
        kisiler[3] = new Student(4, 22, 30.0, "Elektronik");
```

```
        Person p = new Student(5, 15, 10.0, "Bilgisayar");
        p = new Teacher(p.getID(), p.getYas(), p.getBakiye(), 0);
        kisiler[4] = p;

        int i = 1;
        for (Person kisi : kisiler) {
            kisi.yasGuncelle();
            System.out.println(i++ + "---" + kisi);
            kisi.bakiyeGuncelle();
            System.out.println(i + "---" + kisi);
        }
    }
}
```

Çıktı (javac ile denendi). Sondaki hata, derste de görülen boş altıncı konumdan gelir:

```
1---[ID=1, YAS=16, BAKIYE=15.0, BOLUM=Bilgisayar.]
2---[ID=1, YAS=16, BAKIYE=14.5, BOLUM=Bilgisayar.]
2---[ID=2, YAS=41, BAKIYE=15.0, TECRUBE=20.]
3---[ID=2, YAS=41, BAKIYE=14.0, TECRUBE=20.]
3---[ID=3, YAS=56, BAKIYE=25.0, TECRUBE=20.]
4---[ID=3, YAS=56, BAKIYE=24.0, TECRUBE=20.]
4---[ID=4, YAS=23, BAKIYE=30.0, BOLUM=Elektronik.]
5---[ID=4, YAS=23, BAKIYE=29.5, BOLUM=Elektronik.]
5---[ID=5, YAS=16, BAKIYE=10.0, TECRUBE=0.]
6---[ID=5, YAS=16, BAKIYE=9.0, TECRUBE=0.]
Exception in thread "main" java.lang.NullPointerException
```

Beşinci kişi satırı `5---` ile başlayıp `6---` ile devam eder. Hocanın “i zaten sonra 1 arttı, 6 oldu” sözü buna karşılık gelir. Altyazıda okunan değerlerle uyuşma ID 5, yaş 16 ve tecrübe 0 içindir.

18. Örneği Geliştirme Önerisi

Hoca, konuyu öğrenmek için bu koda bir aggregation, composition ya da association ilişkisi eklenmesini önerdi. Örneğin bir `Okul` sınıfı oluşturulur. Öğrencinin bağlı olduğu, öğretmenin çalıştığı okul bilgisi tutulur. Bu ilişki bir association gibi görünür, çünkü arada parça-bütün ilişkisi yoktur. Öğrencinin ya da öğretmenin bilgileri alınırken hangi okulla ilişkili olduğu da gösterilir.

Slaytlardaki hastane örneği de buna benzer: kişi, doktor ve hasta sınıfları, bir de bunlarla ilişkili tedavi (treatment). Doktor ve hasta burada öğretmen ve öğrenciye karşılık gelir. Tedavi ilişkisi de ders alma ya da okul ilişkisine benzer. Hoca, asıl olanın kod yazmak olduğunu, kitaplarda ve internette bu konuda çok örnek bulunduğunu söyledi.

Dersin Toparlaması

Bu derste Akbil örneği üzerinden kalıtımın pratikte nasıl kurulduğu görüldü. Ortak alanlar ve herkes için aynı olan `yasGuncelle()` `Person`’a konuldu. Sınıfa göre değişen `bakiyeGuncelle()` alt sınıflarda override edildi. Üst sınıf tipinde referans dizisi sayesinde farklı sınıflardan nesneler tek döngüde işlendi. Her nesne kendi sınıfındaki metotla çalıştı. Yeni bir kategori eklemek için mevcut sınıflara dokunmak gerekmedi. Yapıcılarda da üst sınıfın önce çalıştığı ve yazılmasa bile gizli bir `super()` bulunduğu gösterildi.

Dersin sonunda üst sınıfta “boşuna duran” metot sorusu abstract kavramına götürdü. Abstract metot gövdesizdir ve alt sınıflara standart koyar. Abstract metodu olan sınıf abstract olmak zorundadır, tersi gerekmez. Abstract sınıftan nesne üretilemez ama referans olarak kullanılabilir. Hoca, abstract konusunun gelecek hafta teorik olarak ayrıntılandırılacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.